

Multiplication ($A \cdot B$) reduces to a single 2-input **AND** gate (2–4 transistors vs. 250+ gates in standard binary multipliers).

Alpha Blending ($\alpha A + (1 - \alpha)B$) reduces to a single 2-to-1 Multiplexer (**MUX**) gate.

To overcome the historical bottlenecks of stochastic computing—specifically **statistical variance noise (salt-and-pepper visual grain)** and **signal correlation errors**—this project leverages **Multi-Dimensional Sobol Quasi-Random Sequences** and deterministic circular bit-rotation matrices. The complete system will be built and evaluated entirely in software using C++ AVX2 SIMD vectorization and Python image profiling tools.

2. Problem Statement & Literature Positioning

The Technical Challenge

1. **The Multiplier Barrier in Edge Vision:** Standard 8-bit binary multipliers require 200–300 logic gates. In energy-constrained edge sensors, micro-drones, and implantable medical vision devices, millions of parallel multiplications deplete battery power rapidly.
2. **The Random Noise Penalty:** Classical stochastic computing uses Linear Feedback Shift Registers (LFSRs) to generate bit-streams. Because LFSRs behave like random coin flips, local bit density fluctuates wildly over short stream lengths ($N \leq 256$), causing severe visual salt-and-pepper artifacts.
3. **Cascading Correlation Failure:** When two stochastic streams originate from shared or correlated random seeds, arithmetic operations fail. For example, squaring a value ($A \cdot A$) by feeding the same stream into both inputs of an **AND** gate yields A instead of A^2 .

Differentiation from Prior Work

Recent literature (e.g., *Moghadam et al., IEEE TNANO 2024*) demonstrates that Low-Discrepancy sequences improve stochastic division by **forcing high stream correlation**. In contrast, this proposed research solves the complementary, open problem in multi-stage color processing: **enforcing strict signal independence for non-linear color math**. This is achieved by mapping orthogonal Sobol dimensions to separate color channels and inserting bit-rotation shift registers between processing stages.

3. Mathematical Foundations

Low-Discrepancy 2D Spatial Coverage

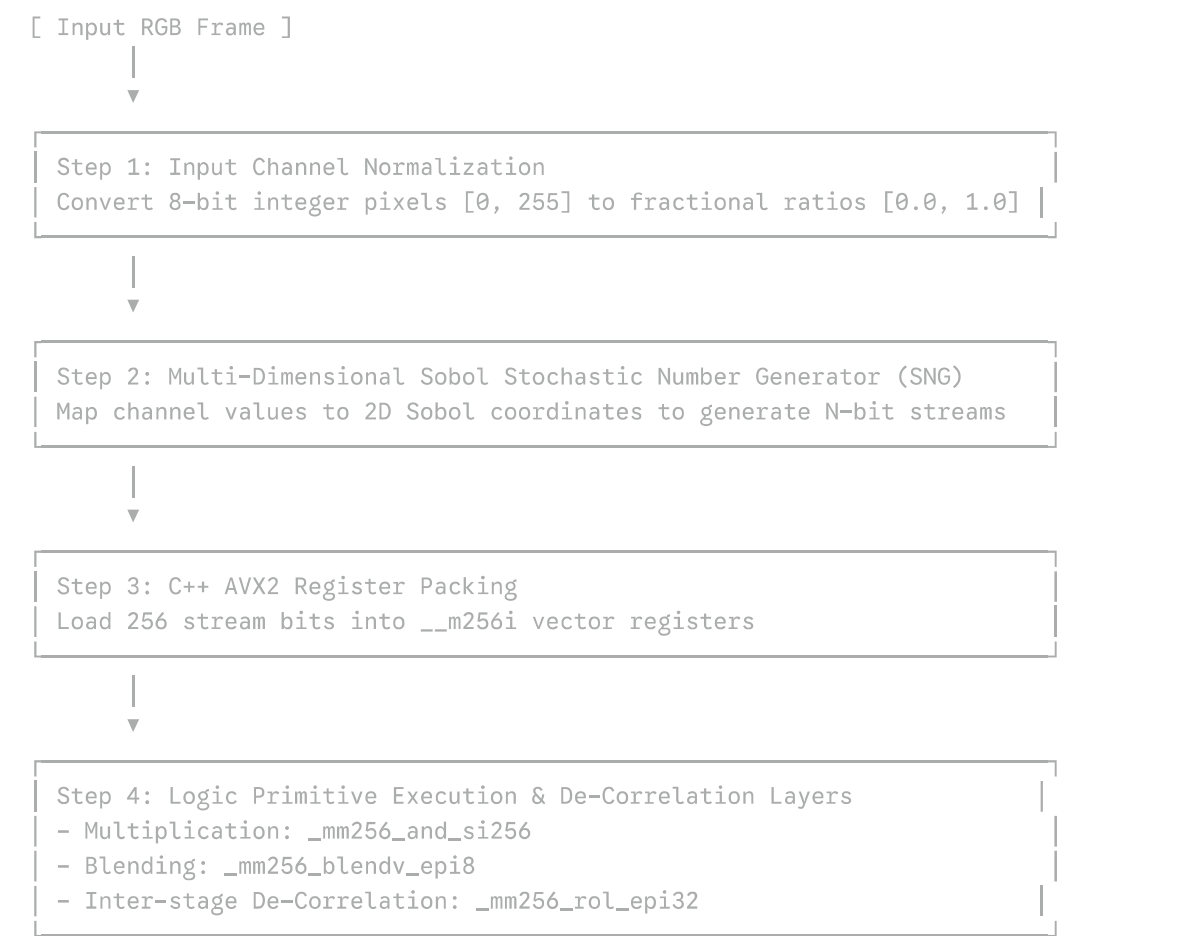
Rather than pseudo-random numbers, this project uses a 2D Sobol sequence generator. To compute the product of two color values $p_A, p_B \in [0, 1]$, the generator outputs a series of 2D coordinates (x_k, y_k) inside a 1×1 unit square:

$$\text{Bit } A_k = \begin{cases} 1 & \text{if } x_k < p_A \\ 0 & \text{otherwise} \end{cases}, \quad \text{Bit } B_k = \begin{cases} 1 & \text{if } y_k < p_B \\ 0 & \text{otherwise} \end{cases}$$

An output 1 from an `AND` gate occurs if and only if $x_k < p_A$ **AND** $y_k < p_B$. The area of this bounding box is $p_A \cdot p_B$. Because Sobol sequences guarantee uniform coverage across binary sub-intervals (Dyadic grid filling), the ratio of output ones over length N matches $p_A \cdot p_B$ with near-zero variance.

Primitive Operation	Target Function	Standard Binary Hardware	Stochastic H
Channel Scaling	$Y = A \cdot B$	8-Bit Multiplier (250+ Gates)	1 AND Gate
Alpha Blending	$Y = \alpha A + (1 - \alpha)B$	2 Multipliers + 1 Adder (600+ Gates)	1 MUX Gate
Color Inversion	$Y = 1 - A$	8-Bit Subtractor (100+ Gates)	1 NOT Gate

4. Step-by-Step Pipeline & System Architecture



Step 5: Reconstruction & Quality Audit

- Accumulate output ones via `_mm256_popcnt`
- Convert back to 8-bit RGB integers
- Evaluate SSIM, PSNR, ΔE^*00 in Python

Detailed Pipeline Steps

- Input Normalization:** Decompose input images (Kodak / McMaster datasets) into RGB color planes and scale pixel intensities to $p \in [0.0, 1.0]$.
- Sobol Bit-Stream Conversion:** Generate uncorrelated bit-streams of length $N \in \{128, 256, 1024\}$ using independent Sobol direction vectors (`boost::random::sobol`).
- AVX2 Vector Packing:** Load 256-bit stochastic vectors directly into C++ AVX2 registers (`__m256i`). This allows a single CPU vector instruction to evaluate **256 simultaneous stochastic operations per clock cycle**.
- Kernel Processing & De-Correlation:**
 - Execute channel weighting via bitwise `AND` (`_mm256_and_si256`).
 - Execute spatial opacity blending via bitwise `MUX` (`_mm256_blendv_epi8`).
 - Apply circular bit-shifts (`_mm256_rol_epi32`) between cascade stages to break signal correlation without generating new random numbers.
- Reconstruction & Evaluation:** Count output ones using hardware population count functions (`_mm256_popcnt`), rescale to $[0, 255]$, and pass output images to Python to compute perceptual quality and hardware gate savings.

5. Work Breakdown & Implementation Roadmap

Week 1-2: Setup SNG & Vector Core	► Week 3-4: Implement Logic Kernels	► Week 5-
• C++ CMake project structure	• AVX2 <code>AND</code> / <code>MUX</code> implementations	• Bit-roi
• Boost Sobol integration	• Popcount reconstruction routines	• Kodak/↑
• Standard LFSR baseline generator	• Verification against unit tests	• Percepti

6. Required Software Toolchain

All development, execution, and profiling occur strictly in software:

Compiler & Build System: GCC 11+ or Clang 13+, CMake 3.18+ (C++17 standard enabled).

C++ Headers & Libraries: `<immintrin.h>` (Intel AVX2 intrinsics), `boost/random/sobol.hpp` (Sobol sequence generator), `libopencv-dev` (Image I/O).

Python Environment: Python 3.10+, `scikit-image` (SSIM, PSNR calculations), `colour-science` (ΔE_{00}^* CIEDE2000 metric), `matplotlib` (data plotting).

Development OS: Ubuntu 22.04 LTS (native or via Windows WSL2).

7. Expected Outcomes & Performance Targets

Evaluation Metric	LFSR Baseline ($N = 256$)	Proposed Sobol Framework ($N = 256$)
Structural Similarity (SSIM)	0.931 (Visible Grain)	≥ 0.990 (High Fidelity)
Color Distance (ΔE_{00}^*)	2.10 (Perceptible Error)	< 0.85 (Imperceptible Error)
Logic Gate Reduction	Baseline Binary Array Multiplier	$> 98.0\%$ Gate Count Savings
SIMD Throughput	1 operation per ALU lane	256 bit-operations per AVX2 register cycle

Primary Deliverables

- 1. A fully vectorized C++17 / AVX2 stochastic image processing engine.
- 2. A Python analytical benchmarking suite generating quality-versus-stream-length trade-off curves.
- 3. A research paper target submission for IEEE Embedded Systems Letters (ESL) or ACM/IEEE Design Automation Conference (DAC).

How would you like to begin implementation?

- Generate C++ AVX2 stochastic kernel implementation↩
- Create CMakeLists.txt and build setup script↩
- Draft the IEEE conference paper template↩

